CIRCULAR SINGLY LINKED LIST

1.insertion at beginning

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* insertBeginning(struct Node *head, int data)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data = data;

    if(head == NULL)
    {
        newNode->next = newNode;
        return newNode;
    }
    struct Node *temp = head;
    while(temp->next != head)
        temp = temp->next;
    temp->next = newNode;
    newNode->next = head;
    head = newNode;
    return head;
}
```

Output

```
10 20 30

=== Code Execution Successful ===
```

```c
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    head = insertBeginning(head, 30);
    head = insertBeginning(head, 20);
    head = insertBeginning(head, 10);
    display(head);
    return 0;
}
```

2.insertion at end

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* insertEnd(struct Node *head, int data)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data = data;

    if(head == NULL)
    {
        newNode->next = newNode;
        return newNode;
    }
    struct Node *temp = head;
    while(temp->next != head)
        temp = temp->next;
    temp->next = newNode;
    newNode->next = head;
    return head;
}
void display(struct Node *head)
```

Output

```
10 20 30

=== Code Execution Successful ===
```

```c
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    head = insertEnd(head, 10);
    head = insertEnd(head, 20);
    head = insertEnd(head, 30);
    display(head);
    return 0;
}
```

3.insertion at given position

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* insertPosition(struct Node *head, int data, int pos)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data = data;
    if(pos == 1)
    {
        if(head == NULL)
        {
            newNode->next = newNode;
            return newNode;
        }
        struct Node *temp = head;
        while(temp->next != head)
            temp = temp->next;
        temp->next = newNode;
        newNode->next = head;
        head = newNode;
        return head;
```

Output

```
5 7 11 9

=== Code Execution Successful ===
```

main.c

```c
    struct Node *temp = head;
    for(int i = 1; i < pos-1; i++)
        temp = temp->next;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    head = insertPosition(head, 5, 1);
    head = insertPosition(head, 7, 2);
    head = insertPosition(head, 9, 3);
    head = insertPosition(head, 11, 3);
    display(head);
    return 0;
}
```

Output

```
5 7 11 9

=== Code Execution Successful ===
```

4.deletion at beginning

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* deleteBeginning(struct Node *head)
{
    if(head == NULL)
        return NULL;
    if(head->next == head)
    {
        free(head);
        return NULL;
    }
    struct Node *temp = head;
    struct Node *last = head;
    while(last->next != head)
        last = last->next;
    head = head->next;
    last->next = head;
    free(temp);
    return head;
}
void display(struct Node *head)
```

Output

```
40 60

=== Code Execution Successful ===
```

main.c

```c
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    // sample nodes
    struct Node *n1 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n2 = (struct Node*)malloc(sizeof(struct Node));
    struct Node *n3 = (struct Node*)malloc(sizeof(struct Node));
    n1->data = 20;
    n2->data = 40;
    n3->data = 60;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = head;
    head = deleteBeginning(head);
    display(head);
    return 0;
```

Output

```
40 60

=== Code Execution Successful ===
```

5.deletion at end

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* deletePosition(struct Node *head, int pos)
{
    if(head == NULL)
        return NULL;
    if(pos == 1)
    {
        struct Node *temp = head;
        struct Node *last = head;
        while(last->next != head)
            last = last->next;
        head = head->next;
        last->next = head;
        free(temp);
        return head;
    }
    struct Node *temp = head;
    struct Node *prev = NULL;
    for(int i=1;i<pos;i++)
    {
```

Output

```
15 10

=== Code Execution Successful ===
```

main.c

```c
    for(int i=1;i<pos;i++)
    {
        prev = temp;
        temp = temp->next;
    }
    prev->next = temp->next;
    free(temp);
    return head;
}
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    struct Node *n3 = malloc(sizeof(struct Node));
```

Output

```
15 10

=== Code Execution Successful ===
```

main.c

```c
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    struct Node *n3 = malloc(sizeof(struct Node));
    n1->data = 15;
    n2->data = 5;
    n3->data = 10;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = head;
    head = deletePosition(head,2);
    display(head);
    return 0;
}
```

Output

```
15 10

=== Code Execution Successful ===
```

6.deletion at given position

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node* deleteEnd(struct Node *head)
{
    if(head == NULL)
        return NULL;
    if(head->next == head)
    {
        free(head);
        return NULL;
    }
    struct Node *temp = head;
    struct Node *prev = NULL;
    while(temp->next != head)
    {
        prev = temp;
        temp = temp->next;
    }
    prev->next = head;
    free(temp);
    return head;
```

Output

```
30 5

=== Code Execution Successful ===
```

main.c

Share Run

```c
    return head;
}
void display(struct Node *head)
{
    if(head == NULL) return;
    struct Node *temp = head;
    do{
        printf("%d ", temp->data);
        temp = temp->next;
    }while(temp != head);
}
int main()
{
    struct Node *head = NULL;
    struct Node *n1 = malloc(sizeof(struct Node));
    struct Node *n2 = malloc(sizeof(struct Node));
    struct Node *n3 = malloc(sizeof(struct Node));
    n1->data = 30;
    n2->data = 5;
    n3->data = 10;
    head = n1;
    n1->next = n2;
    n2->next = n3;
    n3->next = head;
    head = deleteEnd(head);
    display(head);
```

Output

```
30 5

=== Code Execution Successful ===
```